Supplementary Materials

HONGWEI WEI, Harbin Institute of Technology, China

1 Overview

This documentation provides the supplementary materials for the manuscript, entitled **Benchmarking LLM-Based Test Migration from Legacy to Modern Graphics APIs: A Case Study of OpenGL-to-Vulkan**.

A Prompt Template for Compatibility Analysis

The prompt template for the compatibility analysis phase of GL2VKFrame is shown in Fig. 1.

```
You are an expert in graphics API testing. Determine whether Vulkan supports the functionality tested by an OpenGL test based on the summary or source code of the test.
Summary: [SUMMARY] // Add this line for summary-based and hybrid prediction
Source Code: [CODE] // Add this line for code-based and hybrid prediction
## Decision Criteria
Compatible: All test objectives target functionality that is supported by Vulkan.
Incompatible: At least one test objective targets OpenGL-specific functionality that is unsupported by Vulkan, e.g., the OpenGL API with no corresponding Vulkan counterpart, implicit state-machine model behavior, features of shaders unsupported by Vulkan, or the fixed-function rendering pipeline.
## Output Format
Output in the following JSON format:
{
    "compatibility": "xxx", // Compatible or incompatible
    "reason": "xxx" // One sentence explaining the reason of decision
}
```

Fig. 1. The prompt template for compatibility analysis. For readability, we only show a simplified version. The full prompt is available in the current repository.

B Prompt Template for Chain-of-Though Reasoning

The Chain-of-Though prompt template for the single-agent mode of GL2VKFrame is shown in Fig. 2.

C Templates for Test Adapation

The test context construction, Test Case Registration, and Test Configuration templates for the template-based test adaptation phase of GL2VKFrame is shown in Fig. 3, Fig. 4, and Fig. 5, respectively.

Author's Contact Information: Hongwei Wei, weihw@hit.edu.cn, Harbin Institute of Technology, Harbin, China.

You are a graphics test engineer with expertise in OpenGL CTS and Vulkan CTS. Migrate the given OpenGL test into a Vulkan CTS. You are provided with the following JSON object that contains the OpenGL test, the related base test classes, and a reference Vulkan test case.
JSON Object: [JSON]
Reasoning steps
Step 1 - Understanding the OpenGL test case: ...
Step 2 - Understanding the reference Vulkan test case: ...
Step 3 - Analyzing the equivalent test implementation: ...
Step 4 - Generating the Vulkan test implementation: ...
Output Format
Output in the following JSON format:

```
{
    "reasoning_steps": "xxx", // Detailed reasoning steps
    "vulkan_test_code": "xxx" // Source code of the Vulkan test
}
```

Fig. 2. The prompt template of chain-of-thought. For readability, we only show a simplified version. The full prompt is available in the current repository.

```
// external/vulkancts/modules/vulkan/[DEQP_PACKAGE]/vktTestPackage.cpp
# include "vkt[DEQP_GROUP]Tests.hpp"
void TestPackage(void) {
  // ...
  addRootChild("[DEQP_PACKAGE]", createTests);
}

// external/vulkancts/modules/vulkan/[DEQP_PACKAGE]/test_group/vkt
    [DEQP_GROUP]Tests.cpp
# include "vkt[TEST_CASE].hpp"
tcu::TestCaseGroup* createTests(tcu::TestContext& testCtx, const std::string&
    name) {
  // ...
  return createTestGroup(testCtx, name, create[DEQP_GROUP]Groups);
}
void create[DEQP_GROUP]Groups(tcu::TestCaseGroup* group) {
  // ...
  addTestGroup(group, "[DEQP_GROUP]", create[TEST_CASE]Tests);
  create[TEST_CASE]Tests(group);
}
```

Fig. 3. The test context construction template. For readability, we only show a simplified version. The full prompt is available in the current repository.

```
// external/vulkancts/modules/vulkan/{{DEQP_PACKAGE}}/test_case/vkt[TEST_CASE].cpp
# define TEST_CLASS [TEST_CASE] // Use "# define TEST_FUNCTION [TEST_CASE]" when the translated test is a test function.
# define TEST_INPUTS [TEST_INPUTS] // Delete this line if LLM does not generate test outputs.
# define CHECK_SUPPORT [CHECK_SUPPORT] // Delete this line if LLM does not generate "checkSupport" function.
# define INIT_PROGRAMS [INIT_PROGRAMS] // Delete this line if LLM does not generate "initPrograms" function.
[TRANSLATED_TEST] //  Place the source code of the translated test here.
void create[TEST_CASE]Tests(tcu::TestCaseGroup* group) {
# if !defined(INPUT_PARAMS)
# if defined(TEST_CLASS)
  group->addChild(new TEST_CLASS(group->getTestContext(), "[TEST_CASE]"));
# elif defined(TEST_FUNCTION)
# if !defined(CHECK_SUPPORT) && !defined(INIT_PROGRAMS)
  addFunctionCase(group, "[TEST_CASE]", INIT_PROGRAMS);
# elif defined(CHECK_SUPPORT) && !defined(INIT_PROGRAMS)
  addFunctionCase(group, "[TEST_CASE]", CHECK_SUPPORT, TEST_FUNCTION);
# elif !defined(CHECK_SUPPORT) && defined(INIT_PROGRAMS)
  addFunctionCaseWithPrograms(group, "[TEST_CASE]", INIT_PROGRAMS, TEST_FUNCTION);
# elif defined(CHECK_SUPPORT) && defined(INIT_PROGRAMS)
  addFunctionCaseWithPrograms(group, "[TEST_CASE]", CHECK_SUPPORT, INIT_PROGRAMS, TEST_FUNCTION);
# endif
# elif defined(TEST_INPUTS)
  const CaseParams params[] = {TEST_INPUTS};
  for (auto param : params) {
# if defined(TEST_CLASS)
    group->addChild(new TEST_CLASS(group->getTestContext(), "[TEST_CASE]", param));
# elif defined(TEST_FUNCTION)
# if !defined(CHECK_SUPPORT) && !defined(INIT_PROGRAMS)
    addFunctionCase<CaseParams>(group, "[TEST_CASE]", TEST_FUNCTION, params);
# elif defined(CHECK_SUPPORT) && !defined(INIT_PROGRAMS)
    addFunctionCase<CaseParams>(group, "[TEST_CASE]", CHECK_SUPPORT, TEST_FUNCTION, params);
# elif !defined(CHECK_SUPPORT) && defined(INIT_PROGRAMS)
    addFunctionCaseWithPrograms<CaseParams>(group, "[TEST_CASE]", INIT_PROGRAMS, TEST_FUNCTION, params);
# elif defined(CHECK_SUPPORT) && defined(INIT_PROGRAMS)
    addFunctionCaseWithPrograms<CaseParams>(group, "[TEST_CASE]", CHECK_SUPPORT, INIT_PROGRAMS, TEST_FUNCTION, params);
# endif
# endif
  }
# endif
}
```

Fig. 4. The test registration template. For readability, we only show a simplified version. The full prompt is available in the current repository.

```
# external/vulkancts/modules/vulkan/CMakeLists.txt
add_subdirectory({{DEQP_PACKAGE}})
# external/vulkancts/modules/vulkan/[DEQP_PACKAGE]/CMakeLists.txt
set([DEQP_PACKAGE]_SRCS
    vkt[DEQP_GROUP]Tests.cpp
    group/vkt[DEQP_GROUP]Group.cpp
    test_case/vkt[TEST_CASE].cpp)
add_library(deqp-{[DEQP_PACKAGE] STATIC ${[DEQP_PACKAGE]_SRCS})
```

Fig. 5. The test configuration template. For readability, we only show a simplified version. The full prompt is available in the current repository.